
Security Review Report
NM-0993 - Safe Smart Account



NETHERMIND
SECURITY

(August 7, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Safe Smart Account	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Low] getModulesPaginated(...) skips one module between pages	6
6.2	[Info] checkSignatures / checkNSignatures do not bind data to dataHash	6
7	Documentation Evaluation	7
8	Test Suite Evaluation	8
8.1	Tests Output	8
9	About Nethermind	14

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Safe's](#) Smart Account on its 1.3.0 version.

The smart account is the base product, allowing users to create smart contracts that can be used as their accounts. It allows users to manage owners, signature thresholds, execute transactions from the Safe, and extend its functionalities through the use of modules.

The four modules extend the functionalities from the Safe account in the following ways:

- Allowance - allows Safe owners to approve delegates to spend specific assets owned by the Safe.
- Recovery - add the possibility of creating a set of guardians defined for the Safe account, which can rewrite the set of owners for that Safe.
- Passkey - contains contracts that can be deployed as owners of Safe accounts and implement WebAuth for signature verification. This extends the ways Safe accounts can authenticate their owners.
- Safe4337 - Add the necessary mechanisms for the Safe to be a valid ERC-4337 smart account.

The audit comprises approximately 1068 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report two points of attention, where one is classified as Low, and one is classified as Informational. The issue is summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

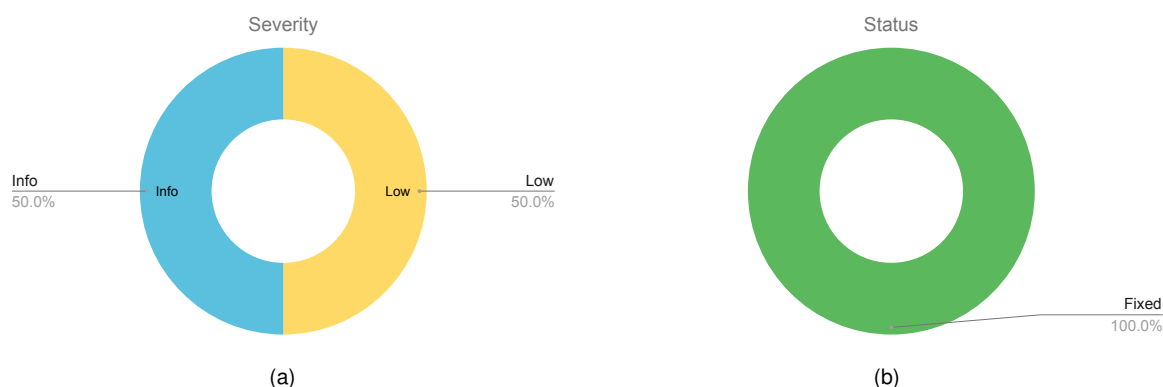


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (1), Undetermined (0), Informational (1), Best Practices (0). Distribution of status: Fixed (2), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Aug 7, 2026
Final Report	Aug 17, 2026
Initial Commit (Safe Account)	767ef36bba88bdb0c9fe3708a4290cabef4c376
Final Commit (Safe Account)	Issues were addressed in v1.4.0
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	contracts/GnosisSafeL2.sol	58	23	39.7%	5	86
2	contracts/GnosisSafe.sol	263	135	51.3%	24	422
3	contracts/handler/DefaultCallbackHandler.sol	50	4	8.0%	7	61
4	contracts/handler/HandlerContext.sol	11	10	90.9%	2	23
5	contracts/handler/CompatibilityFallbackHandler.sol	64	66	103.1%	14	144
6	contracts/accessors/SimulateTxAccessor.sol	37	10	27.0%	5	52
7	contracts/libraries/MultiSend.sol	35	28	80.0%	3	66
8	contracts/libraries/SignMessageLib.sol	17	12	70.6%	5	34
9	contracts/libraries/CreateCall.sol	22	5	22.7%	3	30
10	contracts/libraries/GnosisSafeStorage.sol	12	7	58.3%	2	21
11	contracts/libraries/MultiSendCallOnly.sol	30	30	100.0%	1	61
12	contracts/common/SecuredTokenTransfer.sol	23	11	47.8%	1	35
13	contracts/common/Enum.sol	4	3	75.0%	1	8
14	contracts/common/SignatureDecoder.sol	19	16	84.2%	1	36
15	contracts/common/EtherPaymentFallback.sol	7	4	57.1%	2	13
16	contracts/common/StorageAccessible.sol	22	22	100.0%	3	47
17	contracts/common/Singleton.sol	4	6	150.0%	1	11
18	contracts/common/SelfAuthorized.sol	10	4	40.0%	2	16
19	contracts/external/GnosisSafeMath.sol	24	22	91.7%	8	54
20	contracts/proxies/GnosisSafeProxyFactory.sol	66	32	48.5%	9	107
21	contracts/proxies/GnosisSafeProxy.sol	27	13	48.1%	4	44
22	contracts/proxies/IProxyCreationCallback.sol	10	1	10.0%	1	12
23	contracts/base/OwnerManager.sol	88	49	55.7%	12	149
24	contracts/base/ModuleManager.sol	75	47	62.7%	11	133
25	contracts/base/FallbackManager.sol	33	14	42.4%	6	53
26	contracts/base/GuardManager.sol	36	8	22.2%	6	50
27	contracts/base/Executor.sol	21	5	23.8%	1	27
	Total	1068	587	55.0%	140	1795

3 Summary of Issues

	Finding	Severity	Update
1	getModulesPaginated(...) skips one module between pages	Low	Fixed
2	checkSignatures / checkNSignatures do not bind data to dataHash	Info	Fixed

4 System Overview

Safe is a multisignature smart contract account, it holds ETH, tokens, and arbitrary assets, and executes arbitrary transactions (CALL or DELEGATECALL) only when authorized. Authorization comes from one of two paths: an n-of-m threshold of owner signatures (execTransaction()), or a call from a whitelisted *module* contract, which bypasses signature checks entirely (execTransactionFromModule()). The account is extensible through modules, a transaction *guard*, and a *fallback handler*.

Deployment follows a proxy/singleton pattern: a single GnosisSafe implementation is deployed once per chain, and each user account is a minimal GnosisSafeProxy that stores the singleton address in storage slot 0 and delegatecalls all calls to it. Proxies are deployed by the GnosisSafeProxyFactory, optionally via CREATE2 for counterfactual addresses, and are initialized exactly once through setup(). GnosisSafeL2 is a subclass that only adds events so transactions can be indexed on chains.

4.1 Safe Smart Account

The core account logic is split across the GnosisSafe contract and its base contracts (OwnerManager, ModuleManager, GuardManager, FallbackManager).

The main entry point is execTransaction(). It computes the EIP-712 transaction hash over (to, value, data, operation, safeTxGas, baseGas, gasPrice, gasToken, refundReceiver, nonce) under a domain separator bound to the chain ID and the proxy address, increments the nonce, and calls checkSignatures(). Signatures are packed {r}{s}{v} entries that must be ordered by strictly increasing recovered owner address, four different types of verification exist based on v:

- $v == 0$ - EIP-1271 contract signature: the signer address is encoded in r, and s points to the dynamic signature bytes; validated via isValidSignature() on the owner contract.
- $v == 1$ - approved hash: valid if the owner is msg.sender or previously called approveHash().
- $v > 30$ - eth_sign signature: ecrecover over the digest with the Ethereum signed-message prefix.
- otherwise - standard ECDSA signature over the transaction hash.

If a guard is set, checkTransaction() and checkAfterExecution() hooks run before and after execution and can veto it. A refund mechanism reimburses an arbitrary relayer in ETH or an ERC-20, making the account gas-abstracted.

The remaining relevant functions are only reachable through self-calls because of the authorized modifier:

- setup() - initializer setting owners, threshold, an optional fallback handler.
- addOwnerWithThreshold() / removeOwner() / swapOwner() / changeThreshold() - owner-set management.
- enableModule() / disableModule() - module whitelist management. Enabled modules call execTransactionFromModule(), which executes arbitrary calls with no signatures, no nonce, and no guard checks. Modules are fully trusted.
- setGuard() - sets the guard that will run pre and post transaction.
- setFallbackHandler() - sets the fallback handler. The fallback handler receives all unknown-selector calls via CALL with the original sender appended to calldata.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] getModulesPaginated(...) skips one module between pages

File(s): `contracts/base/ModuleManager.sol`

Description: Enabled modules are stored as a linked list. The mapping `modules` maps each module to the next one, and the `SENTINEL_MODULES` address (`address(0x1)`) marks both the head and the tail of the list. The `getModulesPaginated(...)` function lets a caller read this list in pages. The caller passes `SENTINEL_MODULES` as `start` on the first call, reads the returned array, and then passes the returned next value as `start` on the following call to continue where the previous page ended.

The issue is in how the function chooses where each page begins. It starts iterating from `modules[start]`, meaning the element stored at `start` is used only as a pointer and is never itself added to the returned array.

```

1  function getModulesPaginated(address start, uint256 pageSize)
2      external view returns (address[] memory array, address next)
3  {
4      // ...
5      uint256 moduleCount = 0;
6      // @audit-issue The first item added is modules[start], so start's own next is skipped.
7      address currentModule = modules[start];
8      while (currentModule != address(0x0) && currentModule != SENTINEL_MODULES
9          && moduleCount < pageSize) {
10         array[moduleCount] = currentModule;
11         currentModule = modules[currentModule];
12         moduleCount++;
13     }
14     next = currentModule;
15     // ...
16 }

```

On the first call, this is correct, because `start` is `SENTINEL_MODULES`, which is not a real module, so skipping it is expected. The value assigned to `next` is `currentModule` after the loop, which is the first module that was not included in the current page. That module is an active module that the caller has not seen yet.

On the next call the caller passes this `next` as `start`. The function then begins at `modules[start]`, which is the module after `next`. As a result `next` is never returned in any page. The first module of every page after the first one is silently dropped.

Recommendation(s): Consider checking the pagination logic so that the module referenced by `next` is included at the beginning of the following page.

Status: Fixed

Update from the client: Fixed in future versions, PRs are linked to the following github issue: <https://github.com/safe-fndn/safe-smart-account/issues/461>

6.2 [Info] checkSignatures / checkNSignatures do not bind data to dataHash

File(s): `contracts/GnosisSafe.sol`

Description: The `checkSignatures` and `checkNSignatures` functions verify that a set of signatures satisfies the Safe's owner threshold for a given hash. However, they fail to check that `keccak256(data) == dataHash`. The `data` argument is used only in the EIP-1271 contract-signature branch. The `ecrecover`, `eth_sign`, and `pre-approved-hash` branches verify against `dataHash` alone and ignore `data`.

Consequently, for EOA/approved-hash owners a caller can pass any data with a valid (`dataHash`, `signatures`) pair and the call still succeeds. This doesn't affect the Safe itself, but an external contract that calls these functions and then trusts `data` as "owner-approved" can be fed an arbitrary incorrect data.

Recommendation(s): Consider adding an explicit check so the `data` is bound to the hash. Alternatively, document that a successful call attests only to `dataHash` and signatures, that `data` is used solely for EIP-1271 validation and is otherwise ignored.

Status: Fixed

Update from the client: Fixed in future versions, PRs are linked to the following github issue: <https://github.com/safe-fndn/safe-smart-account/issues/497>

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Safe's documentation

The Safe team provided an overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities. Additionally, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and an in-depth understanding of the project's technical aspects. The Safe team also provided previous audit reports to consult known design choices and later improvements to the protocol.

8 Test Suite Evaluation

8.1 Tests Output

```
> npx hardhat test

SimulateTxAccessor
  estimate
    should enforce delegatecall (512ms)
    simulate call
    simulate delegatecall
    simulate revert

GnosisSafe
  requiredTxGas
    should revert without reason if tx fails (56ms)
    should always revert
    can be called from another contract

GnosisSafe
  execTransaction
    should revert if too little gas is provided (44ms)
    should emit event for successful call execution
    should emit event for failed call execution if safeTxGas > 0
    should emit event for failed call execution if gasPrice > 0
    should revert for failed call execution if gasPrice == 0 and safeTxGas == 0
    should emit event for successful delegatecall execution
    should emit event for failed delegatecall execution if safeTxGas > 0
    should emit event for failed delegatecall execution if gasPrice > 0
    should emit event for failed delegatecall execution if gasPrice == 0 and safeTxGas == 0
    should revert on unknown operation
    should emit payment in success event
    should emit payment in failure event
    should be possible to manually increase gas (45ms)

FallbackManager
  setFallbackManager
    is correctly set on deployment
    is correctly set
    emits event when is set
    is called when set
    sends along msg.sender on simple call
    sends along msg.sender on more complex call

GuardManager
  setGuard
    is not called when setting initially (61ms)
    is called when removed
  execTransaction
    reverts if the pre hook of the guard reverts
    reverts if the post hook of the guard reverts

GnosisSafe
  fallback
    should be able to receive ETH via transfer
    should be able to receive ETH via send
    should be able to receive ETH via call
    should be able to receive ETH via transaction
    should throw for incoming eth with data

ModuleManager
  enableModule
    can only be called from Safe itself
    can not set sentinel
    can not set 0 Address
    can not add module twice
    emits event for new module
    can enable multiple
  disableModule
    can only be called from Safe itself
    can not set sentinel
    can not set 0 Address
```

```

Invalid prevModule, module pair provided - Invalid target
Invalid prevModule, module pair provided - Invalid sentinel
Invalid prevModule, module pair provided - Invalid source
emits event for disabled module
execTransactionFromModule
  can not be called from sentinel
  can only be called from enabled module
  emits event on execution success
  emits event on execution failure
execTransactionFromModuleReturnData
  can not be called from sentinel
  can only be called from enabled module
  emits event on execution failure
  emits event on execution success
  Returns expected from contract on successs
  Returns expected from contract on failure

OwnerManager
  addOwnerWithThreshold
    can only be called from Safe itself
    can not set Safe itself
    can not set sentinel
    can not set 0 Address
    can not add owner twice
    can not add owner and change threshold to 0
    can not add owner and change threshold to larger number than new owner count
    emits event for new owner
    emits event for new owner and threshold if changed
  removeOwner
    can only be called from Safe itself
    can not remove sentinel
    can not remove 0 Address
    Invalid prevOwner, owner pair provided - Invalid target
    Invalid prevOwner, owner pair provided - Invalid sentinel
    Invalid prevOwner, owner pair provided - Invalid source
    can not remove owner and change threshold to larger number than new owner count
    can not remove owner and change threshold to 0
    can not remove owner only owner
    emits event for removed owner and threshold if changed
    - Check internal ownercount state
  swapOwner
    can only be called from Safe itself
    can not swap in Safe itself
    can not swap in sentinel
    can not swap in 0 Address
    can not swap in existing owner
    can not swap out sentinel
    can not swap out 0 address
    Invalid prevOwner, owner pair provided - Invalid target
    Invalid prevOwner, owner pair provided - Invalid sentinel
    Invalid prevOwner, owner pair provided - Invalid source
    emits event for replacing owner
  changeThreshold
    can only be called from Safe itself

GnosisSafe
  setup
    should not allow to call setup on singleton
    should set domain hash
    should revert if called twice
    should revert if same owner is included twice
    should revert if 0 address is used as an owner
    should revert if Safe itself is used as an owner
    should revert if sentinel is used as an owner
    should revert if same owner is included twice one after each other
    should revert if threshold is too high
    should revert if threshold is 0
    should revert if owners are empty
    should set fallback handler and call sub inititalizer
    should fail if sub initializer fails
    should fail if ether payment fails
    should work with ether payment to deployer
    should work with ether payment to account
    should fail if token payment fails
    should work with token payment to deployer

```

should work with token payment to account

GnosisSafe

domainSeparator

should be correct according to EIP-712

getTransactionHash

should correctly calculate EIP-712 hash

getChainId

should return correct id

approveHash

approving should only be allowed for owners

approving should emit event

execTransaction

should fail if signature points into static part

should fail if signatures data is not present

should fail if signatures data is too short

Used 68295 gas for >Execute cancel transaction with EIP-712 signature<

should be able to use EIP-712 for signature generation

should not be able to use different chainId for signing

Used 68467 gas for >Execute cancel transaction with signed Ethereum message<

should be able to use Signed Ethereum Messages for signature generation

Used 64497 gas for >Without pre approved signature for msg.sender<

msg.sender does not need to approve before

if not msg.sender on-chain approval is required

Used 66779 gas for >With pre approved signature<

should be able to use pre approved hashes for signature generation

should revert if threshold is not set

should revert if not the required amount of signature data is provided

should not be able to use different signature type of same owner

Used 84188 gas for >Execute cancel transaction with 4 owners<

should be able to mix all signature types

checkSignatures

should fail if signature points into static part

should fail if signatures data is not present

should fail if signatures data is too short

should not be able to use different chainId for signing

if not msg.sender on-chain approval is required

should revert if threshold is not set

should revert if not the required amount of signature data is provided

should not be able to use different signature type of same owner

should be able to mix all signature types

checkSignatures

should fail if signature points into static part

should fail if signatures data is not present

should fail if signatures data is too short

should not be able to use different chainId for signing

if not msg.sender on-chain approval is required

should revert if not the required amount of signature data is provided

should not be able to use different signature type of same owner

should be able to mix all signature types

should be able to require no signatures

should be able to require less signatures than the threshold

should be able to require more signatures than the threshold

GnosisSafe

follows storage layout defined by GnosisSafeStorage library

StorageAccessible

getStorageAt

can read singleton (41ms)

can read instantiated Safe

simulateAndRevert

should revert changes

should revert the revert with message

should return estimate in revert

Proxy

contructor

should revert with invalid singleton address

ProxyFactory

createProxy

should revert with invalid singleton address (47ms)

should revert with invalid initializer

should emit event without initializing

```

    should emit event with initializing
createProxyWithNonce
    should revert with invalid singleton address
    should revert with invalid initializer
    should emit event without initializing
    should emit event with initializing
    should not be able to deploy same proxy twice
createProxyWithCallback
    check callback is invoked
    check callback error cancels deployment
    should work without callback
calculateCreateProxyWithNonceAddress
    should return the calculated address in the revert message

DebugTransactionGuard
    fallback
        must NOT revert on fallback without value (50ms)
        should revert on fallback with value
    checkTransaction
        should emit debug events

DelegateCallTransactionGuard
    fallback
        must NOT revert on fallback without value (38ms)
        should revert on fallback with value
    checkTransaction
        should revert delegate call
        must NOT revert normal call
        should revert on delegate call via Safe
        can set allowed target via Safe

ReentrancyTransactionGuard
    fallback
        must NOT revert on fallback without value (47ms)
        should revert on fallback with value
    checkTransaction
        should revert if Safe tries to reenter execTransaction
        should be able to execute without nesting

CompatibilityFallbackHandler
    ERC1155
        to handle onERC1155Received (46ms)
        to handle onERC1155BatchReceived
    ERC721
        to handle onERC721Received
    ERC777
        to handle tokensReceived
    isValidSignature(bytes,bytes)
        should revert if called directly
        should revert if message was not signed
        should revert if signature is not valid
        should return magic value if message was signed
        should return magic value if enough owners signed
    isValidSignature(bytes32,bytes)
        should revert if called directly
        should revert if message was not signed
        should revert if signature is not valid
        should return magic value if message was signed
        should return magic value if enough owners signed
    getModules
        returns enabled modules
    getMessageHash
        should generate the correct hash
    getMessageHashForSafe
        should revert if target does not return domain separator
        should generate the correct hash
    simulate
        - can be called for any Safe
        should revert changes
        should return result
        should propagate revert message
        should simulate transaction
        should return modified state

DefaultCallbackHandler

```

```

ERC1155
  should support ERC1155 interface
  to handle onERC1155Received
  to handle onERC1155BatchReceived
ERC721
  should support ERC721 interface
  to handle onERC721Received
ERC777
  to handle tokensReceived
ERC165
  should support ERC165 interface
  should not support random interface

HandlerContext
  parses information correctly
  works with the Safe

GnosisSafe
  0xExploit
    should not be able to use EIP-1271 (contract signatures) for EOA (58ms)
    should revert if EIP-1271 check changes state (46ms)

GnosisSafe
  ERC1155
    should reject if callback not accepted
    should not reject if callback is accepted

GnosisSafe
  Reserved Addresses
    sentinels should not be owners or modules

GnosisSafeL2
  execTransactions
    - should emit SafeMultiSigTransaction event
    - should emit SafeModuleTransaction event

CreateCall
  performCreate
    should revert if called directly and no value is on the factory
    can call factory directly
    should fail if Safe does not have value to send along
    should successfully create contract and emit event
    should successfully create contract and send along ether
  performCreate2
    should revert if called directly and no value is on the factory
    can call factory directly
    should fail if Safe does not have value to send along
    should successfully create contract and emit event
    should successfully create contract and send along ether

GnosisSafeStorage
  follows the expected storage layout

Migration
  constructor
    can not use 0 Address (49ms)
  migrate
    can only be called from Safe itself
    can migrate

MultiSend
  multiSend
    should enforce delegatecall to MultiSend (52ms)
    Should fail when using invalid operation
    Can execute empty multisend
    Can execute single ether transfer
    reverts all tx if any fails
    can be used when ETH is sent with execution
    can execute contract calls
    can execute contract delegatecalls
    can execute all calls in combination

MultiSendCallOnly
  multiSend
    Should fail when using invalid operation (41ms)
  
```

- Should fail when using delegatecall operation
- Can execute empty multisend
- Can execute single ether transfer
- reverts all tx **if** any fails
- can be used when ETH is sent with execution
- can execute contract calls
- can execute combinations

SignMessageLib

- signMessage
 - can only **if** msg.sender provides domain separator
 - should emit event
 - can be used only via DELEGATECALL opcode
 - changes the expected storage slot without touching the most important ones

Upgrade from Safe 1.1.1

- execTransaction
 - should be able to transfer ETH (64ms)
- addOwner
 - should add owner and change threshold
- enableModule
 - should enable module and be able to use it
- multiSend
 - execute multisend via delegatecall
- fallbackHandler
 - should be correctly **set**

Upgrade from Safe 1.2.0

- execTransaction
 - should be able to transfer ETH (66ms)
- addOwner
 - should add owner and change threshold
- enableModule
 - should enable module and be able to use it
- multiSend
 - execute multisend via delegatecall
- fallbackHandler
 - should be correctly **set**

262 passing (4s)

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.